EEL 4742C: Embedded Systems
Name: Ryan Ramdihal
Lab 4: Interrupts & Low-Power Modes

Introduction:

This project uses the capabilities of the MSP430FR6989 microcontroller in controlling LEDs and reading input from push buttons using Timer_A and interrupts. It begins by configuring Timer_A in continuous mode to generate periodic interrupts, which are used to flash an LED. Next, Timer_A is utilized in up mode to generate periodic interrupts and toggle the state of two LEDs. Following that, interrupt-driven input reading from push buttons is implemented to control the LEDs. Finally, the previous sections are modified to utilize low power modes for energy-efficient operation. This lab aims to showcase the versatility and power efficiency of the MSP430FR6989 microcontroller in handling various tasks through hardware interrupts and low power modes.

Part 4.1 Timer's Continuous Mode with Interrupt

In this section, Timer_A will be programmed in continuous mode to generate periodic interrupts, which will trigger an interrupt service routine (ISR) to flash an LED. The ACLK is configured to use a 32kHz crystal oscillator. After setting up Timer_A to run continuously, an ISR is defined to toggle the state of the red LED when the interrupt flag is raised and then reset the flag. The interrupt period is set to occur every 2.048 seconds (65535/32K). Failure to clear the flag after each interrupt will result in the ISR being continuously executed. Meanwhile, the CPU remains in an infinite for-loop in between interrupts.

4.1 Code:

```
// Timer_A continuous mode, with interrupts, flashes LEDs
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
// Configures ACLK to 32 KHz crystal
void config_ACLK_to_32KHz_crystal() {
   // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
   // Reroute pins to LFXIN/LFXOUT functionality
   PJSEL1 &= ~BIT4;
   PJSEL0 |= BIT4;
   // Wait until the oscillator fault flags remain cleared
   CSCTL0 = CSKEY; // Unlock CS registers
   do {
      CSCTL5 &= ~LFXTOFFG; // Local fault flag
      SFRIFG1 &= ~OFIFG; // Global fault flag
   } while((CSCTL5 & LFXTOFFG) != 0); //clears fault flags,then crystal clock ready to use
   CSCTL0_H = 0; // Lock CS registers
return;
}
```

```
void main(void) {
   WDTCTL = WDTPW | WDTHOLD; // Stop the Watchdog timer
   PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
   P1DIR |= redLED; // Configure pin as output
   P9DIR |= greenLED; // Configure pin as output
   P1OUT &= ~redLED; // Turn LED Off
   P9OUT &= ~greenLED; // Turn LED Off
   // Configure ACLK to the 32 KHz crystal
   config_ACLK_to_32KHz_crystal();
   // Configure Timer_A
   // Use ACLK, divide by 1, continuous mode, TAR cleared, enable interrupt for rollback-to-zero event
   TA0CTL =TASSEL_1 | ID_0 | MC_2 | TACLR | TAIE;
   // Ensure the flag is cleared at the start
   TA0CTL &= ~TAIFG;
   // Enable the global interrupt bit (call an intrinsic function)
   _enable_interrupts();
   // Infinite loop... the code waits here between interrupts
   for(;;) {}
   }
   //******* Writing the ISR *******
   #pragma vector = TIMER0_A1_VECTOR // Link the ISR to the vector
   __interrupt void T0A1_ISR() {
      P1OUT ^= redLED;
      TA0CTL &= ~TAIFG;
}
```

Part 4.2 Timer's Up Mode with Interrupt

In this section, we utilize Timer_A in up mode to generate periodic interrupts. The Timer_A module is configured for up mode with interrupts enabled. The TA0CCR0 register is set to 32768, equating to a period of 1 second at a clock frequency of 32 kHz. Additionally, the register is set up to enable interrupts for Channel 0. The interrupt service routine (ISR) toggles the state of both the red and green LEDs each time Timer_A exceeds its maximum value. Consequently, while one LED is illuminated, the other is turned off. To change the delay to 0.5 seconds, the TACCR0 value should equal 16384. TO change the delay to 0.1 seconds , the TACCR0 value should be 3275. The ISR should not clear the flag because the hardware clears it, since it is a non-shared vector.

4.2 Code:

```
// Timer_A up mode, with interrupt, flashes LEDs
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
// Configures ACLK to 32 KHz crystal
void config_ACLK_to_32KHz_crystal() {
   // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
   // Reroute pins to LFXIN/LFXOUT functionality
   PJSEL1 &= ~BIT4;
```

```
    PJSEL0 |= BIT4;
    // Wait until the oscillator fault flags remain cleared
    CSCTL0 = CSKEY; // Unlock CS registers
    do {
       CSCTL5 &= ~LFXTOFFG; // Local fault flag
       SFRIFG1 &= ~OFIFG; // Global fault flag
    } while((CSCTL5 & LFXTOFFG) != 0); //clears fault flags,then crystal clock ready to use
    CSCTL0_H = 0; // Lock CS registers
return;
}
void main(void) {
WDTCTL = WDTPW | WDTHOLD; // Stop the Watchdog timer
PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
P1DIR |= redLED; // Direct pin as output
P9DIR |= greenLED; // Direct pin as output
P1OUT &= ~redLED; // Turn LED Off
P9OUT |= greenLED; // Turn LED On (alternate flashing)
// Configure ACLK to the 32 KHz crystal
config_ACLK_to_32KHz_crystal();
// Configure Channel 0 for up mode with interrupts
TA0CCR0 = 32767;
// 1 second @ 32 KHz, // Delay ~0.5 sec = 16384// Delay ~0.1 sec = 3275
TA0CCTL0 |= CCIE; // Enable Channel 0 CCIE bit
TA0CCTL0 &= ~CCIFG; // Clear Channel 0 CCIFG bit
// Timer_A: ACLK, div by 1, up mode, clear TAR
TA0CTL =TASSEL_1 | ID_0 | MC_1 | TACLR;
// Enable the global interrupt bit (call an intrinsic function)
_enable_interrupts();
for(;;) {}
}
#pragma vector = TIMER0_A0_VECTOR
__interrupt void T0A0_ISR() {
P1OUT ^= redLED;
P9OUT ^= greenLED;
}
```

4.3 Push Button with Interrupt

In this section, we utilize interrupts to read the push buttons. The objective is to toggle the red LED when button S1 is pressed and toggle the green LED when button S2 is pressed. Upon pressing a button, the interrupt service routine is activated. Within this routine, the code determines which button triggered the interrupt and toggles the state of the corresponding LED accordingly. However, there may be instances where the success rate is approximately 50%, meaning that pressing the corresponding button once or twice might be necessary for the LED to toggle.

4.3 code:

```
// Timer_A up mode, with interrupt, flashes LEDs
#include <msp430fr6989.h>
```

```
#define redLED BIT0     // Red LED at P1.0
#define greenLED BIT7   // Green LED at P9.7
#define BUT1 BIT1       // Button S1 at Port 1.1
#define BUT2 BIT2       // Button S2 at Port 1.2
void main(void) {
   WDTCTL = WDTPW | WDTHOLD;   // Stop the Watchdog timer
   PM5CTL0 &= ~LOCKLPM5;       // Enable the GPIO pins
   P1DIR |= redLED;    // Direct pin as output
   P9DIR |= greenLED;  // Direct pin as output
   P1OUT &= ~redLED;   // Turn LED Off
   P9OUT |= greenLED;  // Turn LED On (alternate flashing)
   // Configuring buttons with interrupt
   P1DIR &= ~(BUT1 | BUT2);    // 0: input
   P1REN |= (BUT1 | BUT2);     // 1: enable built-in resistors
   P1OUT |= (BUT1 | BUT2);     // 1: built-in resistor is pulled up to Vcc
   P1IE |= (BUT1 | BUT2);      // 1: enable interrupts
   P1IES |= (BUT1 | BUT2);     // 1: interrupt on falling edge
   P1IFG &= ~(BUT1 | BUT2);    // 0: clear the interrupt flags
   // Enable the global interrupt bit (call an intrinsic function)
   _enable_interrupt();
   for(;;) {}
}
#pragma vector = PORT1_VECTOR   // Write the vector name
__interrupt void Port1_ISR() {
   // Detect button 1 (BUT1 in P1IFG is 1)
   if ((P1IFG & BUT1) != 0) {
      // Toggle the red LED
      P1OUT ^= redLED;
      // Clear BUT1 in P1IFG
      P1IFG &= ~BUT1;
   }
   // Detect button 2 (BUT2 in P1IFG is 1)
   if ((P1IFG & BUT2) != 0) {
      // Toggle the green LED
      P9OUT ^= greenLED;
      // Clear BUT2 in P1IFG
      P1IFG &= ~BUT2;
   }
}
```

4.4 Low Power Modes

In this section, the codes from parts 1 through 3 will be modified to utilize low power modes. The changes involve removing the interrupt call, eliminating the for loop, and engaging the appropriate low power mode for each part. In the first two parts, LPM3 will be used, which deactivates all clocks except for the ACLK utilized in those codes. In the third part, LPM4 will be employed since no clocks were utilized, effectively turning off all clock signals. The low power modes automatically enables the interrupt bit.

4.4.1 Code

```
// Timer_A continuous mode, with interrupt, flashes LEDs
#include <msp430fr6989.h>
#define redLED BIT0     // Red LED at P1.0
#define greenLED BIT7   // Green LED at P9.7
//**********************************
// Configures ACLK to 32 KHz crystal
void config_ACLK_to_32KHz_crystal() {
   // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
   // Reroute pins to LFXIN/LFXOUT functionality
   PJSEL1 &= ~BIT4;
   PJSEL0 |= BIT4;
   // Wait until the oscillator fault flags remain cleared
   CSCTL0 = CSKEY; // Unlock CS registers
   do {
      CSCTL5 &= ~LFXTOFFG;    // Local fault flag
      SFRIFG1 &= ~OFIFG;      // Global fault flag
   } while((CSCTL5 & LFXTOFFG) != 0);
   CSCTL0_H = 0;   // Lock CS registers
   return;
}
void main(void) {
   WDTCTL = WDTPW | WDTHOLD;   // Stop the Watchdog timer
   PM5CTL0 &= ~LOCKLPM5;       // Enable the GPIO pins
   P1DIR |= redLED;    // Direct pin as output
   P9DIR |= greenLED;  // Direct pin as output
   P1OUT &= ~redLED;   // Turn LED Off
   P9OUT &= ~greenLED; // Turn LED Off
   // Configure ACLK to the 32 KHz crystal
   config_ACLK_to_32KHz_crystal();
   // Timer_A configuration (fill the line below)
   // Use ACLK, divide by 1, continuous mode, TAR cleared, enable
   // interrupt for rollback-to-zero event
   TA0CTL = TASSEL_1 | ID_0 | MC_2 | TACLR | TAIE;
   // Ensure the flag is cleared at the start
   TA0CTL &= ~TAIFG;
   _low_power_mode_3();
}
//******* Writing the ISR *******
#pragma vector = TIMER0_A1_VECTOR   // Link the ISR to the vector
__interrupt void T0A1_ISR() {
   // Toggle both LEDs
   P1OUT ^= redLED;
   P9OUT ^= greenLED;
   // Clear the TAIFG flag
   TA0CTL &= ~TAIFG;
}
```

4.4.2 Code:

```
// Timer_A up mode, with interrupt, flashes LEDs
#include <msp430fr6989.h>
#define redLED BIT0     // Red LED at P1.0
```

```
#define greenLED BIT7   // Green LED at P9.7
//*********************************
// Configures ACLK to 32 KHz crystal
void config_ACLK_to_32KHz_crystal() {
   // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
   // Reroute pins to LFXIN/LFXOUT functionality
   PJSEL1 &= ~BIT4;
   PJSEL0 |= BIT4;
   // Wait until the oscillator fault flags remain cleared
   CSCTL0 = CSKEY; // Unlock CS registers
   do {
      CSCTL5 &= ~LFXTOFFG;    // Local fault flag
      SFRIFG1 &= ~OFIFG;      // Global fault flag
   } while((CSCTL5 & LFXTOFFG) != 0);
   CSCTL0_H = 0;   // Lock CS registers
   return;
}
void main(void) {
   WDTCTL = WDTPW | WDTHOLD;   // Stop the Watchdog timer
   PM5CTL0 &= ~LOCKLPM5;       // Enable the GPIO pins
   P1DIR |= redLED;    // Direct pin as output
   P9DIR |= greenLED;  // Direct pin as output
   P1OUT &= ~redLED;   // Turn LED Off
   P9OUT |= greenLED; // Turn LED On (alternate flashing)
   // Configure ACLK to the 32 KHz crystal
   config_ACLK_to_32KHz_crystal();
   // Configure Channel 0 for up mode with interrupt
   TA0CCR0 = (32768 - 1);  // Fill to get 1 second @ 32 KHz
   TA0CCTL0 |= CCIE;       // Enable Channel 0 CCIE bit
   TA0CCTL0 &= ~CCIFG;     // Clear Channel 0 CCIFG bit
   // Timer_A: ACLK, div by 1, up mode, clear TAR (leaves TAIE=0)
   TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLR;
   _low_power_mode_3();
}
//*******************************
#pragma vector = TIMER0_A0_VECTOR
__interrupt void T0A0_ISR() {
   // Toggle the LEDs
   P1OUT ^= redLED;
   P9OUT ^= greenLED;
   // Hardware clears the flag (CCIFG in TA0CCTL0)
}
```

4.4.3 Code:

```
// Timer_A up mode, with interrupt, flashes LEDs
#include <msp430fr6989.h>
#define redLED BIT0     // Red LED at P1.0
#define greenLED BIT7   // Green LED at P9.7
#define BUT1 BIT1       // Button S1 at Port 1.1
#define BUT2 BIT2       // Button S2 at Port 1.2
void main(void) {
```

```
    WDTCTL = WDTPW | WDTHOLD;   // Stop the Watchdog timer
    PM5CTL0 &= ~LOCKLPM5;       // Enable the GPIO pins
    P1DIR |= redLED;    // Direct pin as output
    P9DIR |= greenLED;  // Direct pin as output
    P1OUT &= ~redLED;   // Turn LED Off
    P9OUT |= greenLED;  // Turn LED On (alternate flashing)
    // Configuring buttons with interrupt
    P1DIR &= ~(BUT1 | BUT2);    // 0: input
    P1REN |= (BUT1 | BUT2);     // 1: enable built-in resistors
    P1OUT |= (BUT1 | BUT2);     // 1: built-in resistor is pulled up to Vcc
    P1IE |= (BUT1 | BUT2);      // 1: enable interrupts
    P1IES |= (BUT1 | BUT2);     // 1: interrupt on falling edge
    P1IFG &= ~(BUT1 | BUT2);    // 0: clear the interrupt flags
    _low_power_mode_4();
}
//******************************
#pragma vector = PORT1_VECTOR   // Write the vector name
__interrupt void Port1_ISR() {
    // Detect button 1 (BUT1 in P1IFG is 1)
    if ((P1IFG & BUT1) != 0) {
        // Toggle the red LED
        P1OUT ^= redLED;
        // Clear BUT1 in P1IFG
        P1IFG &= ~BUT1;
    }
    // Detect button 2 (BUT2 in P1IFG is 1)
    if ((P1IFG & BUT2) != 0) {
        // Toggle the green LED
        P9OUT ^= greenLED;
        // Clear BUT2 in P1IFG
        P1IFG &= ~BUT2;
    }
}
```

4.5 Application: Crawler Guidance System

This code implements a program that flashes two LEDs, a red LED and a green LED, using Timer_A in continuous mode with interrupts Additionally, it reads input from two push buttons, S1 and S2. When a button is pressed, the interrupt service routine adjusts the STATE variable accordingly. Button S1 decrements the STATE variable (if greater than -3), while button S2 increments it (if less than 3). Additionally, a delay is added after button presses to debounce the switches.In the Timer_A ISR , the LEDs are toggled based on the current STATE variable. If STATE is 0, both LEDs are toggled simultaneously. If STATE is less than 0, only the red LED is toggled. If STATE is greater than 0, only the green LED is toggled.

4.5 Code:

```
// Timer_A continuous mode, with interrupts, flashes LEDs
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
```

```
#define greenLED BIT7 // Green LED at P9.7
#define BUT1 BIT1 // P1.1
#define BUT2 BIT2 //P1.2
int STATE = 0;
// Configures ACLK to 32 KHz crystal
void config_ACLK_to_32KHz_crystal() {
// By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
// Reroute pins to LFXIN/LFXOUT functionality
PJSEL1 &= ~BIT4;
PJSEL0 |= BIT4;
// Wait until the oscillator fault flags remain cleared
CSCTL0 = CSKEY; // Unlock CS registers
do {
CSCTL5 &= ~LFXTOFFG; // Local fault flag
SFRIFG1 &= ~OFIFG; // Global fault flag
} while((CSCTL5 & LFXTOFFG) != 0);
CSCTL0_H = 0; // Lock CS registers
return;
}
void main(void) {
WDTCTL = WDTPW | WDTHOLD; // Stop the Watchdog timer
PM5CTL0 &=  ~LOCKLPM5; // Enable the GPIO pins
P1DIR |= redLED; // Configure pin as output
P9DIR |= greenLED; // Configure pin as output
P1OUT &=  ~redLED; // Turn LED Off
P9OUT &=  ~greenLED; // Turn LED Off
// Configure the buttons for interrupts
P1DIR &=  ~(BUT1|BUT2); // input
P1REN |= (BUT1|BUT2); // enable built-in resistors
P1OUT |= (BUT1|BUT2); // built-in resistor pull up
P1IES |= (BUT1|BUT2); // interrupt on falling edge (0 for rising edge)
P1IFG &=  ~(BUT1|BUT2); // clear the interrupt flags
P1IE |= (BUT1|BUT2); // enable the interrupts
// Configure Channel 0 for up mode with interrupts
TA0CCR0 = 32767; // 1 second @ 32 KHz
TA0CCTL0 |= CCIE; // Enable Channel 0 CCIE bit
TA0CCTL0 &= ~CCIFG; // Clear Channel 0 CCIFG bit
// Timer_A: ACLK, div by 1, up mode, clear TAR
TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLR;
// Enable the global interrupt bit (call an intrinsic function)
_enable_interrupts();
   for(;;) {
   switch(STATE){
      case -3:
         TA0CCR0 = 16383;
         P9OUT &=  ~greenLED; // Turn LED Off
         break;
      case -2:
         TA0CCR0 = 32767;
         P9OUT &=  ~greenLED;
         break;
      case -1:
         TA0CCR0 = 65535;
         P9OUT &=  ~greenLED;
```

```
            break;
        case 0:
            TA0CCR0 = 32767;
            break;
        case 3:
            TA0CCR0 = 16383;
            P1OUT &=  ~redLED; // Turn LED Off
            break;
        case 2:
            TA0CCR0 = 32767;
            P1OUT &=  ~redLED;
            break;
        case 1:
            TA0CCR0 = 65535;
            P1OUT &=  ~redLED;
            break;
        }
    }
}
//*******************************
#pragma vector = PORT1_VECTOR
__interrupt void Port1_ISR() {
// Action goes here
    //Detect button1
    if((P1IFG & BUT1) == BUT1){
        if (STATE > -3){
        STATE--;
        if (STATE == 0){
            P1OUT &= ~redLED;
            P9OUT &= ~greenLED;
        }
        }
        __delay_cycles (100000);
        P1IFG &=  ~(BUT1);// clear but 1 interrupt flag
    }
    //Detect button2
    if((P1IFG & BUT2) == BUT2){
        if (STATE < 3){
        STATE++;
        if (STATE == 0){
             P1OUT &= ~redLED;
             P9OUT &= ~greenLED;
        }
        }
        __delay_cycles (100000);
        P1IFG &= ~(BUT2); //clear but 2 interrupt flag
    }
}
#pragma vector = TIMER0_A0_VECTOR
__interrupt void T0A0_ISR() {
if (STATE == 0){ //both on
    P1OUT ^= redLED;
    P9OUT ^= greenLED;
}
```

```
if (STATE < 0){
   P1OUT ^= redLED; // turn on if less than
}
if (STATE > 0){
   P9OUT ^= greenLED; //turn on if greater
}
}
```

Conclusion:

In conclusion, this lab highlights the efficiency of the MSP430FR6989 microcontroller in controlling LEDs and handling input from push buttons using interrupts and low power modes. By utilizing Timer_A in continuous and up modes, periodic interrupts were generated to flash LEDs and toggle their states based on button inputs. Additionally, implementing interrupt-driven input reading allowed for responsive interaction with the hardware. The incorporation of low power modes further enhanced energy efficiency by minimizing power consumption during idle periods. Overall, this project demonstrates the effectiveness of hardware interrupts and low power modes in achieving efficient and responsive embedded system designs.

Student Q & A

1. Explain the difference between using a low-power mode and not. What would be the CPU doing between interrupts for each case?

When not using a low-power mode, the CPU remains active between interrupts, consuming power continuously by executing instructions in the main loop. In contrast, when using a low-power mode, the CPU enters a sleep state between interrupts, conserving power by stopping execution until woken up by an interrupt.

2. We're using a module, e.g. the ADC converter, and we're not sure about the vector name. We expect it should be something like ADC VECTOR. Where do we find the exact vector name?

In the microcontrollers data sheet is where you can find the exact vector name.

3. A vector, therefore the ISR, is shared between multiple interrupt events. Who is responsible for clearing the interrupt flags?

If it is a shared vector the programmer clears the flag, if not the hardware clears it.

4. A vector, and its corresponding ISR, is used by one interrupt event exclusively. Who is responsible for clearing the interrupt flag?

The hardware

5. In the first code, the ISR's name is T0A1 ISR. Is it allowed we rename the function to any other name?

Only if the name is changed in the vector.

6. What happens if the ISR is supposed to clear the interrupt flag and it didn’t?
The CPU gets stuck in the interrupt loop.